

AI 가속기와 스토리지의 데이터 이동 병목 분석 및 시스템 최적화

Data Movement-Aware System Architecture for AI Accelerators and Storage

권오영 · 숭실대학교 컴퓨터학부 · 2027년 2월 졸업 예정

데이터가 어떤 형식으로 저장되고 어떤 경로로 이동하는지가 시스템 병목을 만든다. 그 병목을 시뮬레이터 계측으로 측정 가능한 양으로 바꾸고, 통제된 실험으로 검증하고, 가설이 틀렸을 때 그것을 결과로 보고한다.

1. 연구 방향

데이터가 어떤 형식으로 저장되고 어떤 경로로 이동하는지가 시스템 병목을 만든다. 나는 그 병목을 시뮬레이터 계측으로 측정 가능한 양으로 바꾸고, 통제된 실험으로 검증하고, 내 가설이 틀렸을 때 그것을 결과로 보고한다.

지원 직무는 SK하이닉스 Solution SW, 세부적으로는 AI Memory / System Software이며 SSD System Architecture / FTL 영역을 보완 축으로 둔다. 이 문서의 두 프로젝트는 같은 프로젝트가 아니다. 스택의 서로 다른 층에서 **같은 방법론**을 적용한 두 사례다.

	NPU (메인)	SSD (보조)
이동하는 것	LLM KV-cache: 양자화 payload + 그룹별 metadata	Host I/O: LBA write, GC page copy
이동 경로	dequant 엔진 → SRAM → HBM	host queue → FTL mapping → NAND channel
병목 요인	DRAM traffic, dequant 엔진 배분, mode-switch 비용	queue wait, GC-induced stall, write amplification
측정 지표	cycle, DRAM traffic 구성비, 엔진 utilization	p99 latency, WAF, GC stall
도구	계측 확장한 ONNXim (C++, cycle-level)	직접 작성한 FTL/NAND 시뮬레이터 (C++17)

공통 질문은 하나다. **저장 형식과 이동 경로가 병목을 어떻게 만들고, 그것을 어떤 계측으로 증명할 수 있는가.**

2. 지원 직무와 기술 지도

역량	근거	상태
AI workload 분석	KV-cache DRAM traffic 분해, token eviction 실측	verified-historical
Computer architecture	ONNXim dequant 모델·카운터 구현, RV32I RTL	verified-historical
Memory / data movement	배분 민감도 7점 스위치, WAF-OP 곡선	verified-current (두 헤드라인 지점 재현)
Linux / System SW	perf 자동화, Docker 기반 시뮬레이션 파이프라인	documented / 검증됨
Performance profiling	latency 분포(p50/p95/p99), queue wait와 device time 분리	verified-current
SSD / FTL / NAND	page-level FTL-GC 정책 3종·NAND 채널 모델 직접 구현	verified-current
PCIe / NVMe	근거 없음 (not demonstrated)	—
C/C++ · Python	시뮬레이터 코어, 테스트, 실험 자동화	verified-current
Verilog	RV32I single-cycle 통과, 파이프라인 진행 중	documented
재현성	manifest·seed 고정·bit-exact 재현 테스트	verified-current

빈칸을 억지로 채우지 않았다. NVMe/PCIe는 장비가 없으므로 근거가 없다고 적었다.

3. NPU 프로젝트 — 문제 정의

양자화된 LLM에서 KV cache는 균질한 덩어리가 아니다. K와 V는 생성·소비·재참조 시점이 다르고, 각각 payload와 함께 그룹 단위 **양자화 metadata**를 끌고 다닌다. 만약 K와 V가 dequantization 엔진에 구조적으로 다른 수요를 준다면, **하나의 공유 dequant 엔진은 잘못된 자원 배분**이며 K 전용·V 전용으로 쪼개는 편이 낫다.

연구 질문: K와 V의 dequant 자원 수요 차이가 엔진 분할을 정당화할 만큼 큰가, 그렇다면 그 분할은 얼마나 정확히 크기를 맞춰야 하는가.

4. ONNXim 확장 구조

이 질문에 답하려면 시뮬레이터가 먼저 그것을 측정할 수 있어야 했다. 원본 ONNXim에는 dequant 모델도, K/V 분리 카운터도, provenance 로깅도 없었다.

추가한 것	파일	내용
처리량 기반 dequant 비용 모델	SystolicWS.cc	<code>ceil(size_bytes / throughput_B_per_cycle)</code>
shared / split 엔진	SystolicWS.cc	<code>dequant_mode ∈ {disabled, shared, split}</code>
mode-switch 페널티 + 카운터	SystolicWS.cc , Core.h	공유 엔진의 K/V 교대 비용을 가정이 아닌 측정 대상으로
K/V 분리 카운터	Core.h , Core.cc	active/stall cycle, issue count를 K·V로 분리
config 배선	Common.cc , SimulationConfig.h	throughput·demand multiplier·mode
dequant 발행 지점	Attention.cc	QK(K path), PV(V path) 각각
provenance 로깅	main.cc	모든 run이 자기 config와 파라미터를 로그에 남김

설계 시 피해야 했던 함정: ONNXim의 opcode switch는 매치되지 않는 opcode에 대해 **조용히 0 cycle**을 반환한다. DEQUANT 를 enum에만 추가하고 case를 빠뜨리면 크래시 없이 "비용 0"인 깨끗하고 그럴듯하고 완전히 틀린 시뮬레이션이 나온다. 그래서 이 작업은 `dequant_mode=disabled` 대비 회귀 앵커를 항상 함께 돌린다.

5. NPU 실험과 핵심 결과

주 워크로드: llama3-8b-1L-proxy_single , ctx 4096, batch 16, systolic WS 128×128, 4 core, ramulator2 HBM2.

§3-d 오배분 민감도 (demand 80:40 고정, T_shared=128, 7점 스윙)

조건	Total cycle	vs shared
shared 128:128	33,267,145	—
split 110:18	76,810,724	+130.9 %
split 96:32	44,122,913	+32.6 %
split 85:43 (= 수요비)	33,501,237	+0.70 %
split 74:54	38,209,118	+14.9 %
split 64:64	43,878,368	+31.9 %
split 32:96	85,904,231	+158.2 %

여기서 세 가지가 나온다.

- 최적점은 정확히 수요비**이며 1:1, 2:1, 3:1 세 비율에서 모두 성립한다. 한 설정의 우연이 아니다.
- 곡선이 날카롭고 비대칭이다.** 한 스텝만 틀려도 15~33 % 손해이며, K 과다배분 방향이 V 과다배분보다 더 크게 벌점을 받는다. 분할 엔진은 모델·워크로드에 따라 움직이는 수요비에 맞춰야 하는데, 잘 맞춰도 ≤1 % 이득이고 틀리면 30 % 이상 손해다.
- 손익분기에 필요한 전환 비용이 비현실적이다.** 분할이 유리해지려면 공유 엔진의 mode-switch 페널티가 전환 사이 처리량의 약 **0.8~1.2 %**(무차원, 두 모델에서 수렴)에 달해야 한다. datapath mux 재구성 규모를 크게 넘는다.

재현 검증: 두 헤드라인 지점을 이번 세션에 phase0b-kvfix-clean 컨테이너에서 재실행했다.

조건	재실행	아카이브 앵커	판정
shared 128:128	33,267,145	33,267,145	정확히 일치
split 85:43	33,501,237	33,501,237	정확히 일치
델타	+0.70 %	+0.70 %	일치

raw log: results/raw/p0b_sec3d_{A,p3}.log · 재현 명령: ./scripts/reproduce_portfolio_core.sh --mode core (약 12분).

이전 정리 세션은 이 실험의 원본 실행 인자가 어디에도 보존되지 않았다고 기록했으나, 실제로는 §3-d 8개 지점의 sim config 가 랩 호스트에 모두 남아 있었다. 모델 json, models_list, trace(274바이트)까지 전부 저장소로 옮겨 커밋했으므로 이제 재현은 자기완결적이다. 재실행 값이 앵커와 다르면 스크립트는 DISCREPANCY 로 보고하며, 새 값으로 기준을 갈아끼우지 않는다.

6. 가설 기각과 후속 관찰

제안은 자기 실험에 의해 기각됐다. 정확히 맞춘 분할조차 공유 대비 0.7~1.2 % 안쪽이며, 이는 별도 하드웨어를 정당화하지 못한다.

기각 자체보다 중요한 것은 그 뒤에 남은 관찰이다. **W INT4, batch 64** 조건의 DRAM traffic 구성은 다음과 같다.

- weight **21.3 %**
- KV payload **52.4 %**
- **KV metadata 26.3 %**

양자화 metadata가 weight보다 많은 바이트를 옮긴다. 양자화를 더 공격적으로, 더 세밀한 그룹으로 밀수록 이 항은 줄지 않고 커진다. payload를 작게 만들기 위한 장부가 다음 병목이 된다 — 원래 가설이 던지지도 않았던 질문이다.

데이터가 강제한 두 번째 정정: "전환은 드물다(그래서 공유 엔진이 K/V를 몰아 처리한다)"는 사전 가정은 **틀렸다**. llama3-8b 기준 dequant 이벤트의 **98.4 %**가 모드 전환을 동반한다. 공유 엔진이 이기는 결론은 유지되지만, 내가 생각한 이유 때문은 아니었다.

upstream 기여: 작업 중 ONNXim 원본의 `kv_head_idx` GQA/MHA 매핑 버그를 발견해 수정하고 upstream에 보고했다 (브랜치 `fix/kv-head-idx-gqa-mapping @ 69e6189`). 이 버그는 내부적으로도 결정적이었다 — 버그가 있는 빌드의 스팟 체크는 C vs A를 +6.53 %로 보여줬고, 이는 유의성 임계값 5 %를 넘어 **분할 제안을 지지하는 방향**이었다. 수정 빌드로 재실행 하니 +1.13 %로 돌아왔고 제안은 기각됐다. **버그가 내 가설에 유리하게 작동하고 있었다.**

7. SSD 프로젝트 — 문제 정의

Garbage collection은 SSD가 스스로 만드는 tail latency 문제다. free block이 떨어지면 FTL은 **host가 기다리는 동안** valid page를 복사하고 block을 지운다. host에게 이는 "GC"가 아니라 p99 스파이크로 보인다. 흔한 완화책인 free-space threshold 기반 background GC는, host가 바쁠 때 발동하면 stall을 옮길 뿐이고, 옮긴 page는 전부 host가 수명으로 지불하는 write amplification이다.

연구 질문: **host queue 상태를 보는 GC**가, write amplification을 과도하게 키우지 않으면서 GC로 인한 tail latency를 낮출 수 있는가.

트레이드오프 질문이므로 모든 결과를 **tail latency와 WAF의 쌍**으로 보고한다. 무제한의 WAF를 지불해 p99를 낮추는 것은 답이 아니다.

8. FTL/GC 구현 구조

기존 SSD 시뮬레이터를 복사하지 않고 직접 작성했다. mapping/GC/NAND 경로를 남의 모델로 돌리는 게 아니라 내가 모델링 하는 것이 목적이었다.

HostWorkload (trace CSV) → HostQueue queue_depth로 제한, queue wait 누적 → FTL L2P/P2L page-level map, page state, 채널별 write frontier, out-of-place update, old page invalidation, GC policy → NANDModel channel/die/block/page, 채널별 busy-until clock, $t_{\text{read}} 50\mu\text{s}$ · $t_{\text{program}} 600\mu\text{s}$ · $t_{\text{erase}} 3\text{ms}$ → MetricsCollector latency 분포, WAF, GC 카운터, stall, erase 통계

핵심 코어는 약 330줄(src/Ftl.cc)로 의도적으로 작게 유지했다. 중요한 설계 결정은 하나다 — **GC가 queue의 outstanding_io 를 읽는다.** foreground와 fixed-background는 free-page ratio만 본다. 그 추가 입력 하나가 가설 전체 다.

```
``` while free_pages(channel) == 0 or free_ratio(channel) <= critical: force GC now # host가 뒤에서 stall → gc_induced_stall_ns
```

```
if free_ratio(channel) <= background: A foreground: 절대 안 함 B fixed_background: 항상 C queue_aware: outstanding_io <= low_queue_threshold 일 때만 ```
```

free-ratio 검사는 **채널별**이다. free block은 채널 간 이동하지 않으므로 전역 비율은 멀쩡해 보이는데 한 채널만 고갈될 수 있다.

## 9. SSD 실험과 결과

### Experiment 1 — 5 seed, 동일 trace 기준 쌍대 델타 (queue-aware vs foreground)

지표	평균	$\sigma$	seed별
p99 latency	-4.35 %	2.10 %	-5.07, -2.35, -4.56, -7.38, -2.40
GC-induced stall	-12.95 %	0.23 %	-12.91, -12.59, -13.04, -13.03, -13.20
WAF (비용)	+4.15 %	0.10 %	+4.12, +3.98, +4.17, +4.23, +4.24

queue-aware가 모든 seed에서 이기고, stall 감소와 WAF 비용은  $\sigma$ 가 0.1~0.2 %로 매우 안정적이다. 그러나 **p99의 크기는 안정적이지 않다** —  $\sigma$ 가 평균의 절반이다. 따라서 내가 주장하는 것은 "p99가 일관되게 낮아지며 그 크기는 워크로드 의존적"이지 "5 % 빠르다"가 아니다. 이 구분 자체가 결과다.

fixed-threshold background GC는 모든 축에서 파국적으로 나쁘다(WAF 33.3 vs 1.91, p99 25배). queue 상태와 무관하게 일정하게 GC하면 자기가 만든 amplification이 폭주한다. 이 정책의 `gc_induced_stall_ns = 0`은 지표 정의상의 아티팩트(해당 카운터는 host를 막는 GC만 센다)이며, 성과가 아니라 아티팩트로 명시했다.

**Experiment 2~4 (단일 seed, 방향성 결과):** throughput은 `queue_depth = 채널 수(4)`에서 포화한다. WAF는 OP에 따라 단조 감소한다(7/14/28 %에서 2.63 → 1.91 → 1.35). locality는 GC 압력을 크게 줄인다.

**검증:** 12개 invariant가 `ctest` 한 명령으로 돈다 — L2P/P2L 전단사, `free+valid+invalid = 전체`, 채널별 free 합, `overwrite` 무효화, GC 후 데이터 보존, GC 없을 때 WAF = 1.0, GC 시 `nand_writes ≥ host_writes`, 동일 seed-config에서 **bit-exact 재현**, queue depth 상한, critical 압력에서 전진 보장, 잘못된 LBA의 명시적 실패. `CHECK` 매크로는 `abort`가 아니라 실패를 카운트하므로, 통과는 12개가 실제로 다 돌아갔다는 뜻이다.

**실험 중 발견한 버그:** 첫 Experiment 10이 `NAND full`로 죽었다. 원인은 `checkAndRunGc`가 전역 free-page ratio를 임계값과 비교한 것이었다 — free block은 채널별로 분할돼 있으므로 한 채널이 고갈돼도 전역 비율은 건강해 보인다. 세 정책이 모두 지나가는 공유 함수 한 곳에서 고쳤고, 같은 종류의 어긋남이 실험이 아니라 `ctest`에서 잡히도록 채널별 합계 invariant를 추가했다.

## 10. 두 프로젝트의 공통 방법론

---

1. **아키텍처적 직관을 측정 가능한 양으로 바꾼다.** "K와 V는 수요가 다를 것이다" → dequant 엔진 throughput 배분비. "GC가 tail을 만든다" → host를 막은 GC 시간.
2. **측정할 수 없으면 시뮬레이터를 고친다.** ONNXim에 카운터와 비용 모델을 넣었고, SSD 쪽은 아예 시뮬레이터를 썼다.
3. **계측 자체를 먼저 의심한다.** ONNXim의 "0 cycle 조용히 반환" 경로, FTL의 전역 vs 채널별 ratio — 둘 다 그럴듯한 가짜 결과를 만들 수 있는 지점이었고 회귀 앵커와 invariant로 막았다.
4. **결과가 가설을 죽이면 죽인다.** NPU 제안은 기각됐고, GPU 벤치에서 token eviction은 KV 바이트를 약속대로 줄이면서 throughput은 오히려 떨어뜨렸다.
5. **수치보다 산포를 먼저 본다.** 단일 seed의 5 % 차이는 노이즈와 구분되지 않으므로 seed를 늘리고  $\sigma$ 와 함께 보고한다.
6. **모든 숫자에 raw 파일·commit·재현 명령을 붙인다.** 붙일 수 없으면 그렇게 적는다.

## 11. 기술 스택 및 기타 프로젝트

---

- **C/C++17** — FTL 시뮬레이터 코어·테스트, ONNXim(3rd-party, ~10k 라인) 계속 확장
- **Python** — 결정적 trace 생성기, 다중 seed 집계, figure 생성, 실험 오케스트레이션
- **Linux/perf** — Memory Hierarchy Experiment Framework: cache locality, TLB, matrix blocking을 C 마이크로벤치 + perf 자동화로 측정
- **GPU/PyTorch** — KV-cache Consumer GPU Benchmark: RTX 5060/5080에서 KV-cache 메모리 압력·OOM 경계·token eviction 실측. **CUDA 커스텀 커널 경험은 없으며, 있는 척하기 위한 프로젝트를 추가하지 않았다.**
- **Verilog** — RV32I single-cycle CPU가 직접 작성한 testbench를 통과. 파이프라인 버전은 진행 중이며 미완으로 표기한다.
- **재현성 도구** — manifest(seed·인자·config sha256), `reproduce_core.sh` (smoke/core/full), 시뮬레이터 바이너리 provenance 로깅

## 12. 한계와 향후 연구

---

### 정직하게 적는 한계

- SSD 쪽 모든 숫자는 **시뮬레이터 출력**이다. 이 환경에 실물 NVMe가 없고, NAND 타이밍 파라미터는 대표값이지 실측값이 아니다. invariant는 내부 일관성을 검증할 뿐 물리적 정확도를 검증하지 않는다.
- Experiment 1만 seed 스왑이 있다. 2~4는 단일 seed 방향성 결과다.
- 워크로드의 20 ms idle gap이 결과를 지탱한다. idle 창이 없으면 queue-aware는 구조상 foreground와 다를 수 없다. 즉 이것은 **bursty 워크로드에 대한 진술**이다.
- NPU dequant 모델은 타이밍·traffic을 모사하는 **비용 stub**이지 실제 dequant datapath가 아니다. 모델은 single-layer proxy다.
- 정확도(accuracy) 트랙은 미완이다. 이 작업은 cycle과 traffic을 측정하지 양자화가 모델 품질에 주는 영향을 측정하지 않는다.
- HBM 대역폭 민감도는 **의도적으로 돌리지 않았다**. 시뮬레이터의 memory timing 경로에 실제로 반영되는 파라미터인지 검증하지 못한 상태에서 숫자에 배수를 곱하면 근거 없는 그래프가 나온다. Future Work로 남긴다.
- QEMU NVMe 실험은 stretch goal로 잡았다가 **핵심 시뮬레이터 완성도를 위해 제외했다**. 제외 사실 자체를 기록한다.
- RTX 5080은 현재 사용 가능하지 않다. 5080 수치는 과거 측정 아카이브다.

### 향후 연구

1. **KV metadata traffic이 다음 병목**이라는 관찰의 후속 — 그룹 크기·비트폭에 따른 metadata 비중 곡선을 그리고, metadata 압축·재사용의 이득 상한을 계산.
2. 실제 memory bandwidth 파라미터가 timing path에 반영되는지 소스 수준에서 확인한 뒤, context/batch 증가에 따른 stall과 payload/metadata의 대역폭 점유 분리.
3. FTL 쪽: queue-aware 임계값의 자동 튜닝, 그리고 실측 NVMe trace(외부 공개 trace) 로의 재검증. 합성 워크로드에서 얻은 결론은 실제 수요 규모에서 다시 확인해야 한다.